

Memory Allocation for Neural Networks using Graph Coloring

Leonid Barenboim

Department of Mathematics and Computer Science, The
Open University of Israel, Ra'anana, Israel
leonidb@openu.ac.il

Rami Drucker

Vision Business Unit, CEVA DSP Ltd., Herzliya, Israel
Rami.Drucker@ceva-dsp.com

Oleg Zatulovsky

Department of Mathematics and Computer Science, The
Open University of Israel, Ra'anana, Israel
oleg.zat@gmail.com

Eli Levi

Vision Business Unit, CEVA DSP Ltd., Herzliya, Israel
Eli.Levi@ceva-dsp.com

ABSTRACT

The Memory Allocation problem for neural networks can be represented as a two-dimensional optimization problem. The neural network is allocated into limited memory space while allocating as much data as possible into the low latency memory. Our solution is based on a generalization of graph coloring, edge-to-node transformation and considers the order in which the graph nodes are colored. We observed improvement of more than 40% in SRAM memory bandwidth in various neural networks.

CCS CONCEPTS

• **Mathematics of computing**; • **Discrete mathematics**; • **Graph theory**; • **Graph coloring**; • **Software and its engineering**; • **Software organization and properties**; • **Contextual software domains**; • **Operating systems**; • **Memory management**; • **Allocation / deallocation**;

KEYWORDS

Neural Networks, Memory Management, Graph Coloring

ACM Reference Format:

Leonid Barenboim, Oleg Zatulovsky, Rami Drucker, and Eli Levi. 2022. Memory Allocation for Neural Networks using Graph Coloring. In *23rd International Conference on Distributed Computing and Networking (ICDCN 2022)*, January 04–07, 2022, Delhi, AA, India. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3491003.3498354>

1 INTRODUCTION

The use of neural networks in various domains and its use in Deep Learning has recently gained significant superiority in various tasks, such as speech recognition, object detection, classification [1], and more. Many applications use neural networks on edge devices with limited memory capacity [2] which must be utilized efficiently [3]. We consider such devices as drones, smart wearable devices, wireless cameras, etc. These devices are equipped with limited on-board memory that stores neural networks that perform inference tasks. Therefore, efficient memory allocation for neural network components is crucial. We refer to neural networks as directed a-cyclic

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
ICDCN 2022, January 04–07, 2022, Delhi, AA, India
© 2022 Copyright held by the owner/author(s).
ACM ISBN 978-1-4503-9560-1/22/01.
<https://doi.org/10.1145/3491003.3498354>

graphs $G = (V, E)$, where $V = \{v_1, \dots, v_n\}$ is a set of nodes and $E = \{e_1, \dots, e_m\}$ is a set of weighted and directed edges. A node in the graph represents a mathematical operation and a weighted edge represents a data buffer (the weight on the edge is the buffer size) which acts as an input and an output for its neighboring nodes. During network inference, each of the data buffers is accessed for reading or writing. Given a node in the graph, all its input and output buffers must be in different memory addresses, or otherwise we would end up with data being overwritten. The problem of allocating data buffers between two (or more) types of memory can be reduced to a graph coloring problem in which coloring a buffer means allocating it into a specific memory type such as SRAM (low latency) or DDR (high latency). Previous work [4], which dealt with a related but somewhat different setting, achieved 11% reduction in memory footprint. This was achieved using a different strategy than ours, namely, by a strategy for sharing buffers among intermediate tensors in deep neural networks.

2 COLORING PROBLEM GENERALIZATION

Edge to node transformation. The first step in our solution is transforming a graph $G = (V, E)$ into a new graph $\tilde{G} = (\tilde{V}, \tilde{E})$. Since we want to color nodes, every weighted edge in E is transformed into a node in $\tilde{V}$, while keeping the weight as the buffer size. The set $\tilde{E}$ is built by transforming every $v_i \in V$ to edges in $\tilde{E}$. The idea is to connect every pair of nodes in $\tilde{V}$ in such a way that their consequent buffers in $\tilde{G}$ could not have identical colors (located in separate memories) under proper coloring.

Algorithm 1 GREEDY ALGORITHM: Highly optimal buffers allocation in a limited size memory

1. Run edge-to-node transformation // $G \rightarrow \tilde{G}$
Sort and queue the nodes in $\tilde{G}$ according to a given order.
2. Initialize: $SRAM_{COLORS}$ // Set of colors assigned to SRAM
For each node $\tilde{v}_i$ in the coloring queue:
 - 2.1 For each color c_k in $c \in \{C \cup c_{j+1}\}$: // New color c_{j+1}
 - 2.1.1. While $\tilde{v}_i$ is colored with color c_k :
 - 2.1.1.1. Find and record the colors set u_i which, if assigned to SRAM, produces the largest SRAM bandwidth and lowest SRAM cost in consequent priority while $SRAM_{Cost} < SRAM_{Size}$.
 - 2.2 Node $\tilde{v}_i$ is colored with color c_k , which produces u_i with the highest recorded SRAM bandwidth and the lowest SRAM cost in consequent priority.
 - 2.3 Update $SRAM_{COLORS}$ with u_i .

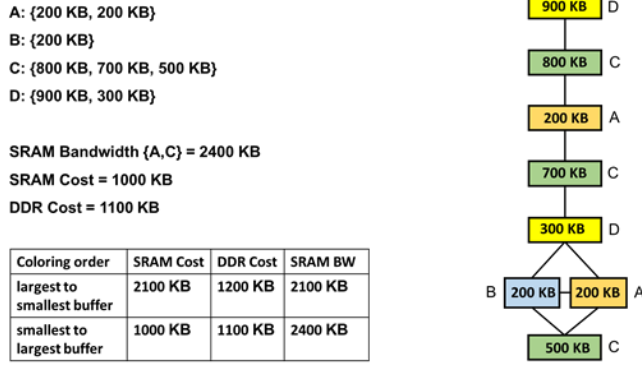


Figure 1: Processed buffers graph. Color classes A, C are assigned with SRAM for maximal bandwidth. The coloring order significantly affects the result. SRAM size is 1000 KB.

Greedy coloring. We represented the memory allocation task for neural networks by a generalized version of the classical graph coloring [5] problem. Once $\tilde{G}$ is constructed, we apply the coloring algorithm under a specific coloring order [6]. We investigated several orderings that turned out to be close to optimal in our previous studies: increasing vertex degree, decreasing vertex degree, increasing buffer size, decreasing buffer size. The highest SRAM bandwidth improvement is achieved by selecting the best out of these four coloring orderings. We define the bandwidth of SRAM to be the sum of all the buffer sizes that are assigned to SRAM (when a color is assigned to SRAM, all the buffers with that color are in SRAM). The cost of SRAM is the sum of the largest buffers within each color class that is assigned to SRAM. At every iteration we greedily maximize the SRAM bandwidth and minimize SRAM cost as demonstrated in Figure 1. The proposed greedy algorithm colors all the nodes using $|c|$ colors in $O(n|c|2^{|c|})$ time complexity. Although $2^{|c|}$ is exponential in the number of colors, the long and narrow structure of neural networks allows us to use very few colors. All our experiments showed that the final number of colors is significantly lower than the number of nodes $n \gg |c|$.

Real-Time scenario. Prior to the inference stage, the proposed algorithm efficiently utilizes the limited SRAM memory space for buffers allocation. During network inference, the same nodes are calculated over and over in the same order and maximal amount of data flows faster through SRAM memory, improving inference speed and real-time experience. Although some platforms use more than two memory hierarchies, the proposed algorithm will utilize the faster memories first.

3 FINAL RESULTS

The proposed algorithm was tested against a previously used algorithm on a CEVA-XM6 processor simulator with 2MB SRAM memory. Figure 2 shows the memory bandwidth and cost compared to the algorithm that is currently used by CEVA Inc.

In some networks we reached above 40% increase in SRAM bandwidth and above 50% decrease in DDR (high latency) bandwidth. SRAM cost demonstrates highly efficient utilization (closer to SRAM

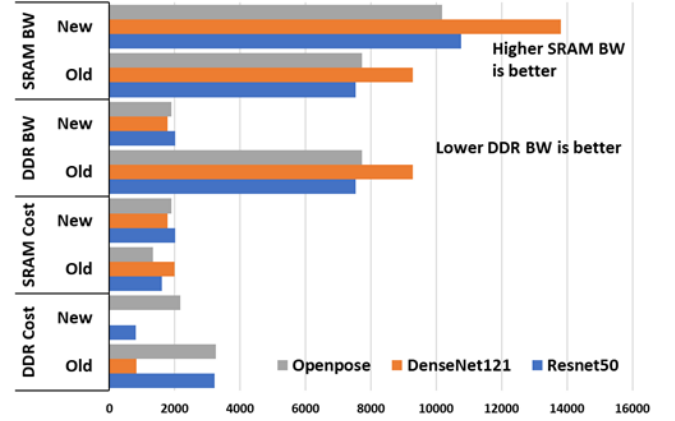


Figure 2: Old vs. New algorithm. Bandwidth and cost of three neural networks. We seek higher SRAM BW and lower DRAM BW. SRAM size is 2 MB. DDR size is 500 MB.

size) of the SRAM memory space. Some networks achieved optimal allocation and the SRAM was optimally used. In most cases, increasing buffer size order achieved the best SRAM bandwidth result. Our future work considers dynamic memory management and multiple neural network inference on a single memory space.

ACKNOWLEDGMENTS

This research was supported by the Israel Innovation Authority.

REFERENCES

- [1] A. Krizhevsky, I. Sutskever and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Advances in Neural Information Processing Systems*, pp. 1097-1105, 2012.
- [2] J. Zhang, S. H. Yeung, Y. Shu, B. He and W. Wang, "Efficient memory management for gpu-based deep learning systems," arXiv:1903.06631, 2019.
- [3] S. V. Pemmaraju, R. Raman, and K. Varadarajan. "Buffer minimization using max-coloring," *SODA*, vol. 4, 2004.
- [4] Y. Pisarchyk and J. Lee, "Efficient Memory Management for Deep Neural Net Inference," arXiv preprint arXiv:2001.03288, 2020.
- [5] L. Li, H. Feng and J. Xue, "Compiler-directed Scratchpad Memory Management via Graph Coloring," *ACM Trans. Archit. Code Optim.*, vol. 6, no. 3, pp. 1-17, 2009.
- [6] H. A. Kierstead, "A polynomial time approximation algorithm for dynamic storage allocation," *Discrete Mathematics*, vol. 88, pp. 231-237, 1991.